

UNIVERSIDAD MARIANO GÁLVEZ DE GUATEMALA



Logo UMG

ANÁLISIS DE SISTEMAS II

Richard Ortiz

Arquitectura Repository

Módulo 20: «Validación de resultados por bioquímico» sobre el repositorio del equipo

GERSON GIOVANNI ORELLANA VÉLIZ

Carnet: 1890-23-7082

Miércoles 19 de agosto de 2026

Índice

1. Introducción
2. Ficha técnica y consigna
3. Arquitectura del módulo en el repositorio del equipo
 - 3.1. Vista de componentes
 - 3.2. Patrón Repository y dependencias
 - 3.3. Propiedad del módulo y multitenencia
4. Reglas de dominio y casos de uso
5. Persistencia en PostgreSQL
6. Presentación (API REST)

7. Pruebas automatizadas e integración
8. Evidencia de ejecución
9. Conclusión
10. Bibliografía

1. Introducción

Esta tarea integra el módulo **Validación de resultados por bioquímico (ASII-20)** en el **repositorio del equipo** `compilations-teams/sistema-hospitalario-integrado-SistemasII-2026`, aplicando el patrón **Repository** como estilo de arquitectura. El flujo es el mismo modelado desde la Tarea 1: el bioquímico **revisa, valida o rechaza** un resultado de laboratorio; un resultado validado es **inmutable**; un rechazo exige **motivo obligatorio**; y una corrección crea una **versión nueva** en estado pendiente.

La arquitectura se organiza en **cuatro capas** (Domain, Application, Infrastructure y Presentation) separadas por puertos y adaptadores: el caso de uso depende de la interfaz `LabResultRepository` y de la abstracción `AutorizadorValidacion`; las implementaciones concretas (Eloquent sobre PostgreSQL, Spatie para permisos) se inyectan en el contenedor. El módulo es **propiedad del hospital** (HOSPITAL) dentro del modelo federado CENTRAL/HOSPITAL y accede a los datos del CENTRAL por **identificadores lógicos** (UUID de tenant), nunca con claves foráneas remotas.

Todos los datos usados son ficticios y no contienen información clínica identificable.

2. Ficha técnica y consigna

Campo	Descripción
Universidad / Curso	Análisis de Sistemas II — 2026
Estudiante	GERSON GIOVANNI ORELLANA VÉLIZ
Carnet	1890-23-7082
GitHub	gioore
Módulo oficial	Validación de resultados por bioquímico
Repositorio evaluado	https://github.com/compilations-teams/sistema-hospitalario-integrado-SistemasII-2026
Rama de trabajo	feat/mod20-validacion-resultados-bioquimico (creada desde origin/develop)
Entregables	Docs de módulo, 5 diagramas UML, código vertical, migración/seeder, tests (dominio, aplicación, integración PostgreSQL), EVIDENCIA.md, DECLARACION_IA.md, PR

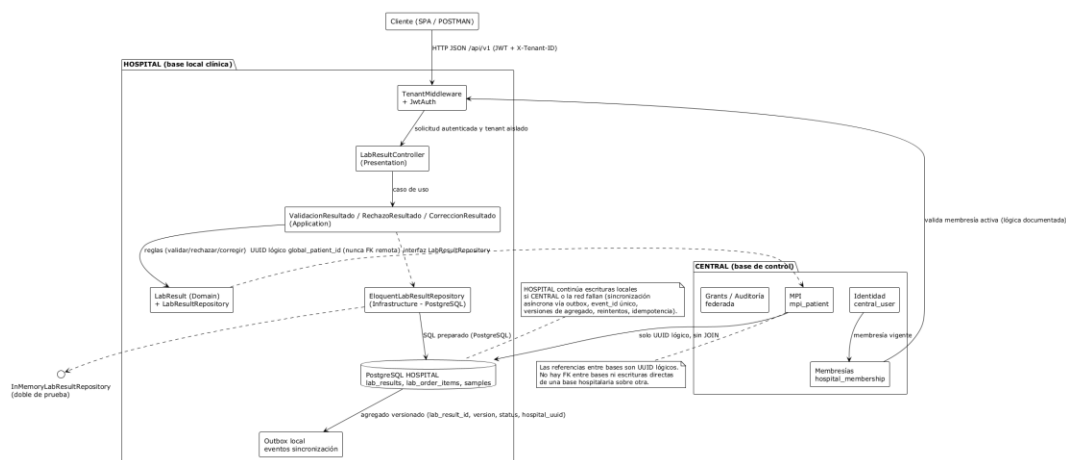
Campo	Descripción
Requisitos técnicos	borrador, informe PDF Laravel 12 + PostgreSQL; arquitectura de capas; patrón Repository; ≥ 10 commits; migrate: fresh --seed funcionando; tests ≥ 3 dominio, ≥ 2 aplicación, ≥ 1 integración PostgreSQL

Declaración de datos. Todos los datos, nombres de pacientes, valores y resultados son **ficticios**. No se incluye información clínica identificable real.

Nota sobre control de versiones. Por indicación explícita del equipo, el trabajo quedó preparado en el worktree local **sin subir nada a Git** durante la elaboración de esta evidencia. La rama está lista para commits, push y el PR borrador hacia develop en cuanto el usuario lo autorice.

3. Arquitectura del módulo en el repositorio del equipo

3.1. Vista de componentes



Vista de componentes

Fuente editable: componentes.puml

El módulo 20 se implementa como un **núcleo de dominio puro** bajo `app/Domain/Modulo20`, un **caso de uso de aplicación** bajo `app/Application/Modulo20` y una **capa de infraestructura** bajo `app/Infrastructure/Modulo20`, con el controlador REST en `app/Http/Controllers/Api/V1`. Las reglas de negocio no conocen a Laravel, a la base de datos ni al HTTP.

3.2. Patrón Repository y dependencias

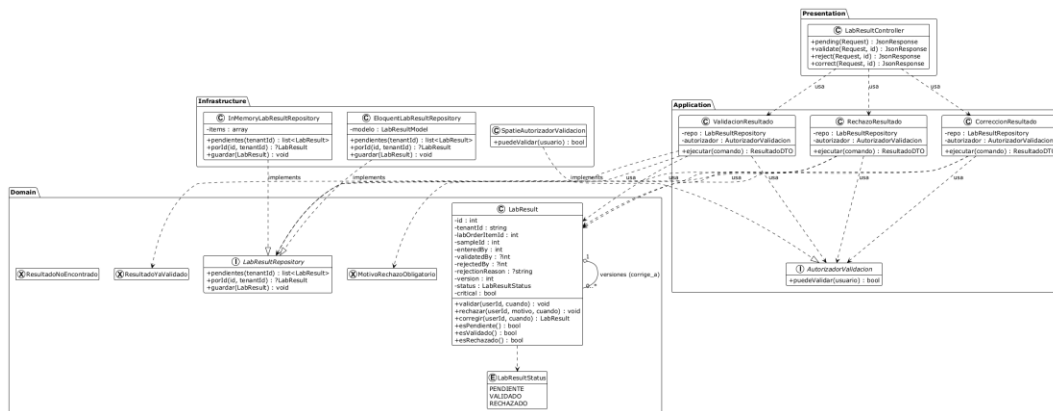


Diagrama de clases

Fuente editable: `diagrama-clases.puml`

El caso de uso `ValidacionResultado` depende de la **interfaz** `LabResultRepository` (puerto), no de una implementación concreta. Se aplica el estilo **Repository**:

Abstracción (puerto)	Implementación (adaptador)
LabResultRepository (Domain)	EloquentLabResultRepository (PostgreSQL)
LabResultRepository (Domain)	InMemoryLabResultRepository (tests)
AutorizadorValidacion (Application)	SpatieAutorizadorValidacion (permiso <code>lab.results.validate</code>)

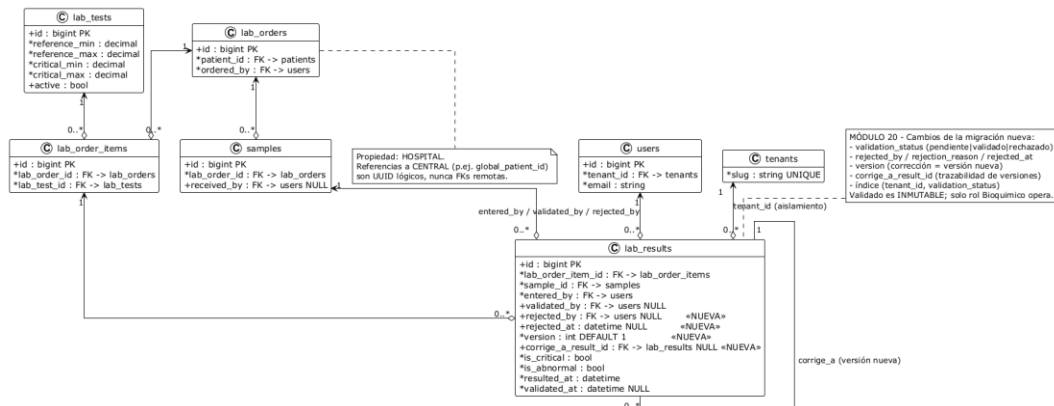
Los bindings se registran en `AppServiceProvider`:

```
$this->app->bind(LabResultRepository::class, EloquentLabResultRepository::class);
$this->app->bind(AutorizadorValidacion::class, SpatieAutorizadorValidacion::class);
```

La interfaz está **orientada al lenguaje del dominio** (no es un CRUD genérico):

```
interface LabResultRepository
{
    /** @return list<LabResult> */
    public function pendientes(string $tenantId): array;
    public function porId(int $id, string $tenantId): ?LabResult;
    public function guardar(LabResult $resultado): void;
    public function proximoId(): int;
}
```

3.3. Propiedad del módulo y multitenencia

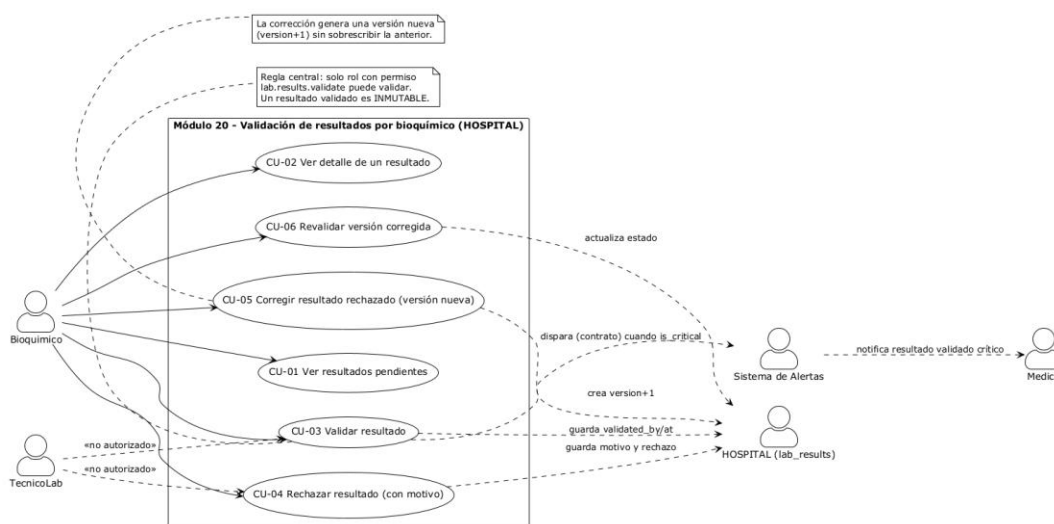


Vista de datos

Fuente editable: vista-datos.puml

- El módulo 20 es **propiedad del hospital** (capa HOSPITAL). Las tablas lab_tests, lab_orders, samples y lab_results pertenecen al hospital.
- El tenant se identifica con un **UUID lógico** (tenant_id, char(36)) que referencia al CENTRAL; **no hay claves foráneas remotas**.
- Todos los accesos del repositorio filtran por tenant_id (WHERE tenant_id = ? AND ...), lo que aísla los datos de cada hospital (ver CA-07).

4. Reglas de dominio y casos de uso



Casos de uso

Fuente editable: casos-de-uso.puml

Regla

Caso de uso

Código HTTP

Regla	Caso de uso	Código HTTP
Solo el rol autorizado (Bioquímico) valida o rechaza	validar / rechazar	403
Solo un resultado pendiente puede validarse	validar	200
Un resultado validado es inmutable	doble validación	409
El motivo del rechazo es obligatorio	rechazar	422
La corrección crea una versión nueva pendiente	corregir	201
Resultado inexistente o de otro tenant	cualquier acción	404 / 403

Las secuencias de validación y rechazo se documentan en `secuencia-validar.puml` y `secuencia-rechazar.puml`.

La entidad `LabResult` encapsula estas reglas (métodos `validar()`, `rechazar()`, `corregir()`), y las excepciones de dominio (`ResultadoYaValidado`, `MotivoRechazoObligatorio`, `ResultadoNoEncontrado`) las traducen a respuestas HTTP por el controlador.

5. Persistencia en PostgreSQL

La implementación real `EloquentLabResultRepository` usa Eloquent sobre la conexión **PostgreSQL** del proyecto (clúster local `127.0.0.1:5433`, base `shi_asii20`). La migración `2026_08_19_000001_add_validation_to_lab_results.php` agrega al esquema existente:

Columna	Propósito
<code>validation_status</code> enum(pendiente,validado,rechazado)	estado de validación
<code>rejected_by</code> FK nullable	quién rechazó
<code>rejection_reason</code> string(500)	motivo obligatorio
<code>rejected_at</code> datetime	cuándo se rechazó
<code>version</code> unsignedInteger default 1	versionado por corrección
<code>corrige_a_result_id</code> FK self nullable	traza de la corrección
índice <code>idx_results_validation_pending</code>	panel de pendientes del bioquímico

Comando verificado:

```
php artisan migrate:fresh --seed
```

Resultado: 90 resultados sembrados en estado pendiente, rol Bioquimico con permiso lab.results.validate.

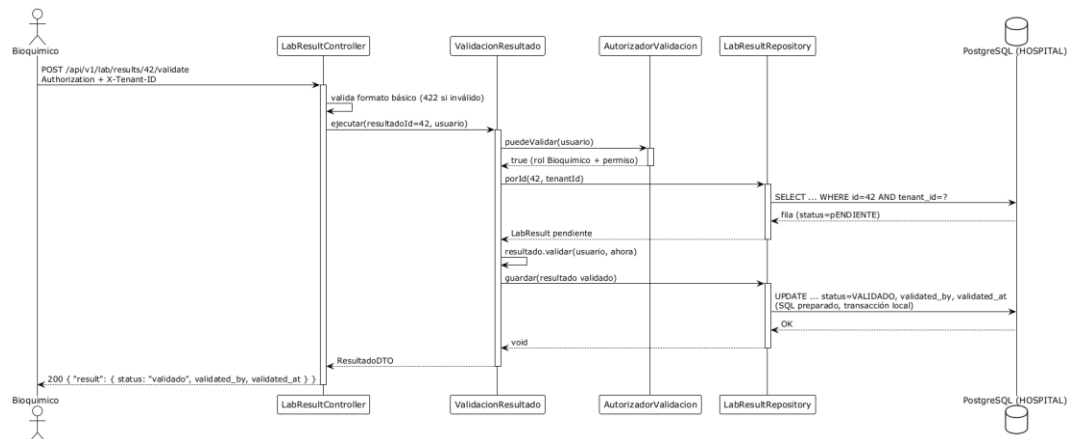
6. Presentación (API REST)

Ruta	Método	Descripción	Códigos
/api/v1/lab/results/pending	GET	panel de pendientes	200
/api/v1/lab/results/{id}/validate	POST	validar	200, 403, 404, 409
/api/v1/lab/results/{id}/reject	POST	rechazar con motivo	200, 403, 404, 409, 422
/api/v1/lab/results/{id}/correct	POST	corregir (versión nueva)	201, 403, 404

Middleware aplicado: tenant (valida X-Tenant-ID), jwt.auth (JWT del equipo) y permission:lab.results.validate (Spatie). En bootstrap/app.php se configuró que las rutas api/* respondan siempre JSON (incluye los errores de validación 422).

El controlador LabResultController es delgado: valida la entrada, construye el **command**, invoca el caso de uso y traduce excepciones a respuestas. No contiene SQL ni reglas de negocio.

7. Pruebas automatizadas e integración



Secuencia de validación

Fuente editable: secuencia-validar.puml

php artisan test --filter="Modulo20"

PASS Tests\Unit\Modulo20\LabResultDomainTest

✓ un resultado pendiente puede validarse

(6)

- ✓ un resultado validado es inmutable
- ✓ el rechazo exige un motivo no vacío
- ✓ el rechazo con motivo marca el resultado como rechazado
- ✓ la corrección crea una versión nueva pendiente
- ✓ un resultado rechazado no puede validarse directamente

PASS Tests\Feature\Modulo20\ValidacionResultadoUseCaseTest (5)

- ✓ un usuario sin permiso no puede validar
- ✓ un usuario autorizado valida un resultado pendiente
- ✓ el rechazo sin motivo se rechaza en el caso de uso
- ✓ un resultado no encontrado en el tenant lanza error
- ✓ la corrección genera una versión nueva desde un rechazado

PASS Tests\Feature\Modulo20\EloquentLabResultRepositoryIntegrationTest (2)

- ✓ el repositorio persiste una validación en postgresql
- ✓ la corrección persiste una versión nueva en postgresql

Tests: 13 passed (30 assertions)

Duration: 1.38s

Cobertura solicitada: **6 de dominio** (≥ 3), **5 de aplicación** (≥ 2) y **2 de integración**

PostgreSQL (≥ 1 , contra la base shi_asii20_test del clúster real).

8. Evidencia de ejecución

Prueba funcional HTTP completa (servidor php artisan serve --port=8000):

#	Caso	Respuesta
1	GET /lab/results/pending (bioquímico)	200, lista de pendientes
2	POST /lab/results/33/validate (técnico sin permiso)	403
3	POST /lab/results/33/validate (bioquímico)	200 , validated_by=4, validado
4	POST /lab/results/33/validate (repetición)	409 inmutable
5	POST /lab/results/34/reject sin motivo	422
6	POST /lab/results/34/reject	200 , rechazado

#	Caso	Respuesta
	con motivo	
7	POST /lab/results/34/correct	201 , nuevo id=93, version=2, pendiente
8	POST /lab/results/999999/validate	404
9	POST /lab/results/35/validate con tenant ajeno	403

Los 9 casos coinciden con los criterios de aceptación CA-01 a CA-09 de docs/modulos/mod20/ESPECIFICACION.md. El detalle completo, las salidas reales y el inventario de archivos están en docs/modulos/mod20/EVIDENCIA.md; la declaración de uso de IA está en docs/modulos/mod20/DECLARACION_IA.md.

9. Conclusión

Qué se logró. El módulo 20 quedó integrado en el repositorio del equipo como una vertical completa de capas con patrón **Repository**: reglas de dominio puras, casos de uso dependientes de puertos, adaptadores Eloquent/PostgreSQL y Spatie, presentación REST con JWT y permisos, migración y seeders verificados con `migrate:fresh --seed`, y **13 pruebas** que pasan, incluidas **2 de integración contra PostgreSQL real**.

Decisión más relevante. Modelar la interfaz LabResultRepository con el vocabulario del dominio (pendientes, porId, guardar, proximoId) en lugar de un repositorio CRUD genérico. Eso hizo que el caso de uso expresara la intención de negocio y que las pruebas sustituyeran la persistencia por el doble InMemoryLabResultRepository sin tocar producción (doble uso del estilo Repository: desacoplar la persistencia y habilitar tests).

Limitación que permanece. Por orden explícita del equipo no se ejecutó la etapa Git (commits ≥10, push y PR borrador) durante la elaboración; la rama quedó lista en el worktree. Además, la notificación al técnico y la alerta crítica a médicos (CU-05/CU-06 de la Tarea 1) quedaron como piezas futuras del módulo.

Cómo la evidencia demuestra el cumplimiento. El repositorio contiene la especificación y el ADR, 6 diagramas UML editables y en PNG, el código vertical, la migración y seeders, las pruebas ejecutadas con salida real, EVIDENCIA.md, DECLARACION_IA.md y este informe PDF. Todo es reproducible con `composer install, php artisan migrate:fresh --seed, php artisan serve --port=8000` y `php artisan test`.

10. Bibliografía

- Object Management Group. *Unified Modeling Language (UML), versión 2.5.1*.

- Fowler, Martin. *Patterns of Enterprise Application Architecture* (Repository, 2002).
- Documentación oficial de Laravel. «Database», «Eloquent ORM» y «Validation». <https://laravel.com/docs>
- Documentación oficial de PostgreSQL 17. <https://www.postgresql.org/docs/>
- Tarea 1 del curso. *Diagramas UML del módulo ASII-20* (casos de uso y secuencias).
- Tarea 2 del curso. *Aplicación de DIP* al flujo de validación por bioquímico (MVP Cluster).
- Tarea 3 del curso. *Micro-HIS: micro-monolito en capas* (PHP vanilla, PDO preparado).